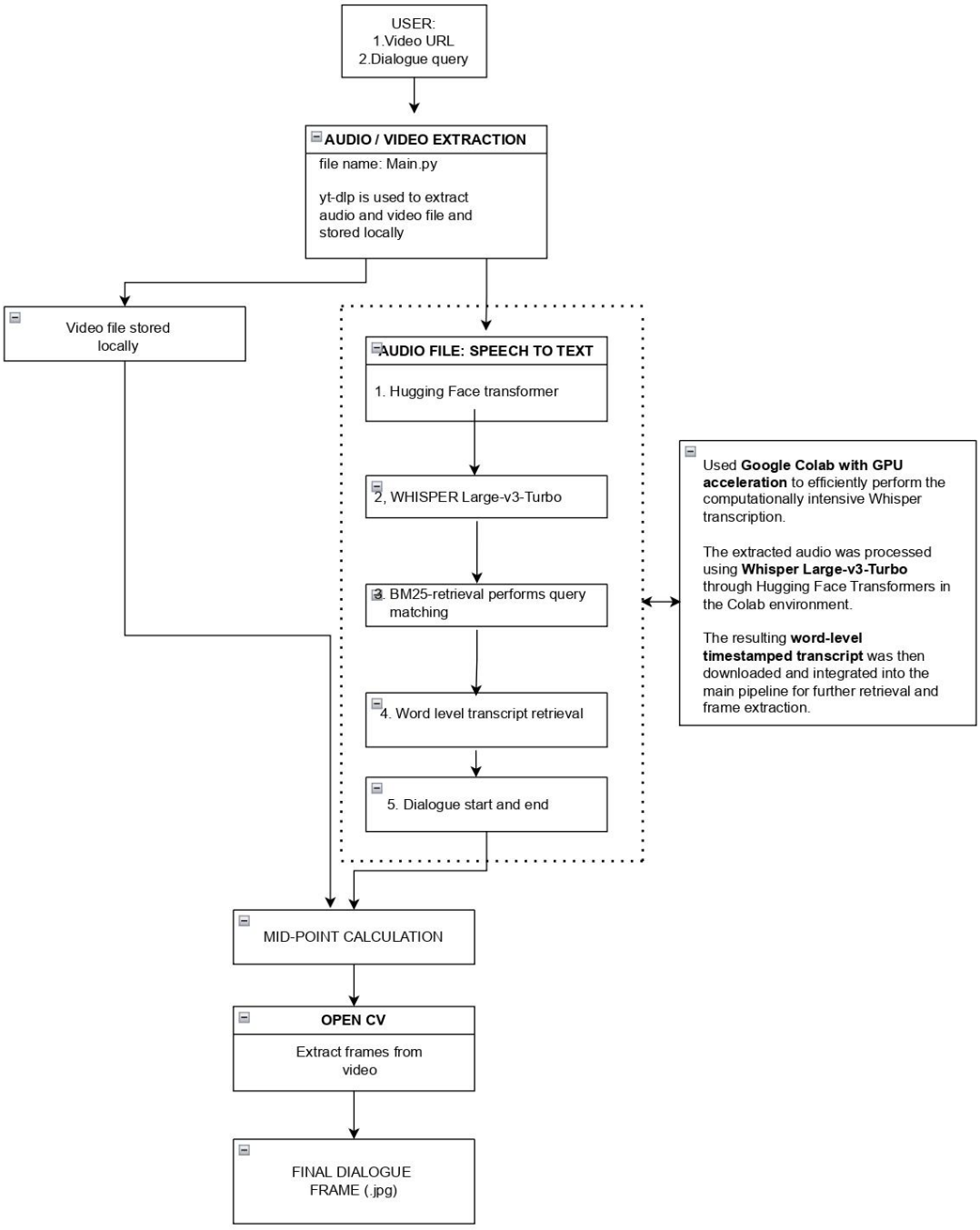
Design and Approach

System Architecture

The project is designed as a **modular, sequential pipeline** that converts a user's video URL and dialogue query into a relevant video frame. The architecture is divided into separate stages, with each stage responsible for one specific part of the problem. This makes the system easier to understand, test, debug, and modify.



1. **AUDIO/VIDEO EXTRACTION:**

The process begins when the user provides a video URL.

- The video is downloaded locally using **yt-dlp**, and the downloaded media provides both the **video and audio files** that are store locally in downloads folder.
- The video is preserved because it will later be used for frame extraction, while the audio is used for speech recognition.

2. TEXT EXTRACTION FROM AUDIO:
   - The audio is passed to **Whisper Large-v3-Turbo from Hugging Face**, which converts the spoken content into text while retaining word-level timestamp information.
   - The model is accessed through the **Hugging Face Transformers library**, allowing us to integrate the pretrained Whisper model directly into the transcription workflow.
   - During the model selection stage, we created **three separate branches to evaluate different Whisper-based transcription approaches** and compared their performance for our video-dialogue retrieval requirements.
   - Based on the results of this evaluation, we concluded that **Whisper Large-v3-Turbo** provided the most suitable balance of transcription accuracy, timestamp precision, and computational efficiency for our system, and therefore selected it for the final pipeline.
   - The timestamps are essential because the system needs to identify not only what was spoken, but also the exact location of that dialogue within the video.
   - Since Whisper is computationally demanding, **Google Colab with GPU acceleration** is used to perform transcription efficiently. T
   - The generated word-level timestamped transcript is then downloaded and brought back into the main project pipeline for further processing and retrieval.

3. **LEXICAL MATCHING AND TIMESTAMP EXTRACTION:**
   - The transcript is subsequently **preprocessed and indexed using BM25**.
   - Preprocessing normalizes the text by handling differences such as capitalization, punctuation, whitespace, and tokenization.
   - BM25 is then used to search the transcript based on the user's query.
   - It was selected because the problem primarily involves finding dialogue containing specific words from the video, making lexical retrieval effective while remaining lightweight and fast.
   - After BM25 identifies the most relevant transcript segments, the system uses their associated **timestamps to locate the relevant portion of the original video**.
   - A word-level matching component can then be used to refine the result by finding the exact sequential occurrence of the requested dialogue within the timestamped transcript. This gives the system a more precise start and end time for the spoken statement.

4. **FRAME EXTRACTION:**
   - Once the dialogue interval has been identified, the system moves from **text retrieval to visual processing**. A representative timestamp is selected within the identified dialogue range.
   - The current implementation uses the midpoint of the range because it provides a simple, deterministic point that is less likely to systematically favour the beginning or end of the dialogue.

- The selected timestamp is then passed to **OpenCV**, which accesses the original downloaded video and extracts the corresponding frame. The extracted image is saved as the final output.
- Since the system already knows where the dialogue occurs, it does not need to analyze every frame of the video, significantly reducing unnecessary visual processing.